Task Ranking and Allocation Heuristics for Efficeint Workflow Schedules

Kuo-Chan Huang
Department of Computer Science
National Taichung University of Education
Taichung, Taiwan
kchuang@mail.ntcu.edu.tw

Meng-Han Tsai
Department of Computer Science
National Taichung University of Education
Taichung, Taiwan
darg20127@gmail.com

***Abstract*—Task ranking and allocation are two major steps in list-based workflow scheduling. This paper explores various possibilities, evaluates recent approaches in the literature, and proposes several new task ranking and allocation heuristics. A series of simulation experiments have been conducted to evaluate the proposed heuristics. Experimental results indicate that effectiveness of task ranking and allocation heuristics largely depends on the characteristics of workflows to be scheduled, and our new scheduling heuristics can outperform previous methods when dealing with workflows of high CCR (Communication-to-Computation Ratio) values.**

Keywords: workflow scheduling, task ranking, task allocation, communication-to-computation ratio

I. INTRODUCTION

Workflow scheduling has long been an important research topic [1][2][3]. Since it is a challenging NP-complete problem, many scheduling heuristics have been proposed for tackling it. Most heuristic methods for workflow scheduling can be classified into three categories [6]: list-based, clustering-based, and duplication-based. List-based heuristics sort all the tasks of a workflow into a list according to specific prioritization mechanisms and then schedule the tasks following the order in the list [1][2]. The main idea of clustering-based heuristics [12][13] is to reduce communication delay by allocating the tasks of heavy communication onto the same processor. On the other hand, duplication-based heuristics [11] try to reduce the communication cost for a task to transmit data to its succeeding task(s) by duplicating it on the processors of those tasks.

Among the three categories, list-based scheduling has the advantages of simplicity and broader applicability. For example, it can be effectively extended to schedule mix-parallel workflows or concurrent workflows where the other two categories of heuristics cannot be directly applied. Clustering-based heuristics cannot deal with mixed-parallel workflows, and duplication-based heuristics are not appropriate for concurrent workflow scheduling since task duplication consumes extra resources and thus degrades the execution performance of all concurrent workflows.

Therefore, in this paper, we focus on exploring list-based workflow scheduling, which consists of two major steps: task ranking and task allocation. Many previous list-based heuristics have been proposed [1][2][6], which adopt different methods for these two steps. Heterogeneous Earliest Finish Time (HEFT) is one of the most well-known listed based heuristics because of its good performance and low complexity, and was shown to outperform previous approaches in terms of both performance and cost metrics in [6]. A lookahead variant of HEFT was proposed in [2], which makes decision of a task' allocation taking into consideration the impact of such allocation on the finish time of its children. This approach leads to better workflow schedules than HEFT at the cost of higher algorithm complexity. A most recent state-of-the-art list-based scheduling algorithm called Predict Earliest Finish Time (PEFT) was proposed in [1], which has the same time complexity as HEFT [6] while outperforming it and its lookahead variant [2] significantly by introducing an effective look-ahead feature.

In this study, we explore the possibility of several new task ranking and allocation heuristics, and evaluate their effectiveness in comparison with the state-of-the-art method, PEFT [1]. A series of simulation experiments have been conducted to evaluate the proposed heuristics. Experimental results show that the performance of task ranking and allocation heuristics largely depends on the characteristics of workflows. Our new scheduling heuristics outperform PEFT [1] when scheduling workflows of high CCR (Communication-to-Computation Ratio) values.

The rest of this paper is organized as follows. Section II discusses the three most relevant works in list-based workflow scheduling. Section III presents the new task ranking and allocation heuristics, and illustrates their effectiveness. The proposed scheduling heuristics are evaluated with a series of simulation experiments in Section IV. Section V concludes this paper.

II. PREVIOUS LIST-BASED SCHEDULING ALGORITHMS

In most previous works [1][2][3][6], workflows are represented by Directed Acyclic Graphs (DAG) for describing the inter-task precedence constraints. Figure 1 is an example of such kind of workflow structure. Each node represents a task which can be realized by a specific software program and can be allocated to a processor for execution. The number next to each node indicates the required execution time of the task. The edges represent the dependence between tasks, and the number next to an edge means the inter-task data transmission cost. Node A is called the entry node and node M is the exit node. A workflow scheduling algorithm has to schedule each task according to the dependence specified in the workflow definition.

978-1-5090-3438-3/16 $31.00 © 2016 IEEE
DOI 10.1109/ICS.2016.107

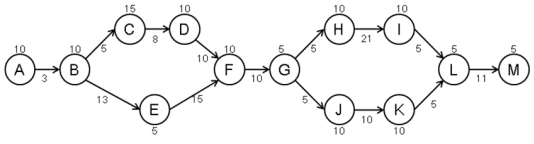


Figure 1: An example workflow of DAG structure

List-based workflow scheduling consists of two major steps: task ranking and task allocation. In the task ranking step, the scheduling algorithm assigns a rank value to each task according to a specific mechanism, and then sorts the tasks into a list according to their rank values. In the task allocation step, the scheduling algorithm continuously allocates each task, following the order in the list, onto an appropriate processor for execution according to a specific processor selection mechanism. Different list-based scheduling heuristics differ in the mechanism used in these two steps, and thus lead to different schedules.

A. Heterogeneous Earliest Finish Time (HEFT)

HEFT [6] is one of the most famous list-based workflow scheduling approaches. In the task ranking step, HEFT computes the rank value of each task based on its computation and communication costs together with the dependence on other tasks. Then, it sorts all tasks into a list with decreasing order of the rank value. The rank of each task, called $rank_u$, is computed recursively from the exit node in a workflow. The rank value of a task n_i is defined as the following formula,

$$rank_u(n_i) = \overline{w_i} + \max_{n_j \in succ(n_i)} (\overline{c_{i,j}} + rank_u(n_j)). \quad (1)$$

where $\overline{w_i}$ is the computation cost of task n_i, $succ(n_i)$ indicates the successors of task n_i, and $\overline{c_{i,j}}$ is the communication cost between tasks n_i and n_j. The rank value of the exit node is its computation time. In the task allocation step, each task is allocated to the processor that minimizes its Earliest Finish Time (EFT).

B. Lookahead Variant of HEFT

The main idea of the lookahead variant of HEFT in [2] is to improve HEFT by a new heuristic which makes task allocation decisions not only considering a single task, such as HEFT, but also looking ahead in the schedule and taking into account the impact of a decision on the children of the task being allocated. The lookahead variant adopts the same task ranking mechanism as HEFT, i.e. equation (1).

Four scheduling alternatives were proposed in [2] as follows.

1) Lookahead

The processor selected for a task is the one which minimizes the maximum EFT of all its children which are scheduled using HEFT.

2) Lookahead with weighted average EFT

The processor selected for a task is the one which minimizes the average EFT of all its children, where each child's EFT is weighted by its rank value.

3) Lookahead with priority change

Each time, two ready tasks are chosen for consideration with the lookahead information for the children of both tasks. After comparing the maximum EFT of both tasks' children, the task with smaller children's maximum EFT will be scheduled first.

4) Lookahead with weighted average EFT and priority change

This alternative adopts the same mechanism for priority change as the previous one, but uses the weighted average EFT instead of maximum EFT.

C. Predict Earliest Finish Time (PEFT)

The lookahead variant of HEFT [2] produces better workflow schedules than HEFT [6], but has higher computational complexity since it has to schedule a task's all children first before making the task allocation decision. PEFT [1] is a most recent state-of-the-art list-based scheduling algorithm, which achieves better schedules than the lookahead variant [2] while maintaining the same lower computational complexity as HEFT [6].

PEFT computes an Optimistic Cost Table (OCT) before the task ranking and allocation steps. The OCT is a matrix where each row indicates a task and each column stands for a processor. Each entry in OCT represents the maximum length of the shortest paths from a task's children to the exit node, considering a specific processor is selected for allocating it. The OCT value of task t_i on processor p_k is defined recursively as follows. The OCT value of the exit node on each processor is set to zero.

$$\mathrm{OCT}(t_i, p_k) = \max_{t_j \in succ(t_i)} \left[\min_{p_w \in P} \{ OCT(t_j, p_w) + w(t_j, p_w) + \overline{c_{i,j}} \} \right], \ \overline{c_{i,j}} = 0 \ if \ p_w = p_k. \quad (2)$$

In the task ranking step, PEFT uses the average of OCT values on all processors as a task's rank value, called $rank_{oct}$. In the task allocation step, each task is allocated to the processor with minimum Optimistic EFT (O_{EFT}). The O_{EFT} of task t_i on processor p_k is defined as follows, which is the sum of the earliest finish time of t_i and its OCT value on p_k.

$$O_{EFT}(t_i, p_k) = \mathrm{EFT}(t_i, p_k) + \mathrm{OCT}(t_i, p_k). \quad (3)$$

Both PEFT [1] and the lookahead variant of HEFT [2] try to estimate the impact of a task's allocation on its children. However, instead of computing the estimate dynamically and repeatedly during the task allocation step as the lookahead variant of HEFT [2], PEFT computes the OCT before the task ranking and allocation steps for just once, resulting in lower computational complexity. Moreover, the lookahead variant of HEFT [2] considers only a task's immediate successors, while PEFT [1] takes into account a task's all successors till the exit node, leading to better workflow schedules.

III. Our New Scheduling Heuristics

This section presents new task ranking and allocation heuristics, which are promising in further improving workflow schedules.

A. min-max OCT

We called the OCT in PEFT [1] a max-min OCT since it chooses the best case when considering processors and selects the worst case regarding a task's successors, as indicated in equation (2). In general, there might be four possibilities for computing OCT, i.e. max-min, max-max, min-min, and min-max. PEFT adopts the max-min mechanism without discussing the reason behind. In the following, we explore another possibility, called min-max OCT as defined in equation (4), which chooses the worst case when evaluating processors and selects the best case among a task's successors.

$$OCT_{min-max}(t_i, p_k) = \min_{t_j \in succ(t_i)} \left[\max_{p_w \in P} \{ OCT(t_j, p_w) + w(t_j, p_w) + \overline{c_{i,j}} \} \right], \ \overline{c_{i,j}} = 0 \ if \ p_w = p_k \quad (4)$$

The following is an example illustrating how min-max OCT could outperform max-min OCT. Figure 2 is an example workflow containing 18 tasks and its CCR being 10. The workflow is to be scheduled on three processors. Table 1 shows the computation time of each task on each processor. Figure 3 illustrates the workflow schedule produced by PEFT using max-min OCT [1], and Figure 4 is the schedule resulted from using our min-max OCT. The difference between these two schedules starts at the third task to schedule. PEFT using max-min OCT [1] tends to schedule task 8 first, while our min-max OCT allows task 2 to run before task 8. In general, max-min OCT would choose the task containing the longest path to the exit node to run first, i.e. task 8 in this example. On the other hand, our min-max OCT would tend to schedule the task with more ahead workload, including both computation and communication costs, first, i.e. task 2 in this example, resulting in a better schedule.

B. Allocated Top Rank and OCT

In the following, we evaluate another heuristic for the task ranking step. The top rank of a task in a workflow is recursively defined as the following equation in [5], simply based on the static structural properties of a workflow. Moreover, the allocated top rank of a task is a dynamic property based on a partial schedule, where the parents of the task have all been scheduled. Therefore, the communication cost between any pair of two tasks would be omitted when calculating the allocated top rank value if the two tasks are allocated to the same processor in the partial schedule.

$$\mathrm{TR}(t_i) = \begin{cases} 0, & if \ \text{task} \ t_i \ is \ entry \ task \\ \max\limits_{t_j \in pre(t_i)} \left(\overline{c_{i,j}} + TR(t_j) \right), & otherwise \end{cases} \quad (5)$$

In this new heuristic, a task's rank value is defined to be the summation of its allocated top rank and the average of its OCT values on all processors. The allocated top rank of each task has to be updated each time a new ready task is scheduled. Compared to the previous approaches, HEFT [6], the lookahead variant [2], and PEFT [1], this new heuristic tends to give higher priority values to the tasks on the allocated critical paths, promising in leading to better schedules. This heuristic has two variants since the allocated top rank can be accompanied with either max-min OCT or min-max OCT. Figures 5 and 6 are examples of this heuristic's two variants illustrating how the new heuristic achieves better schedules, compared to PEFT [1]. In these two examples, our new heuristic schedules task 2 before task 8, in contrast to the schedule by PEFT in Figure 3, resulting in earlier finish of task 7 and thus the entire workflow.

Table 1 Computation time of tasks on each processor

	Processor 0	Processor 1	Processor 2
Task 0	119	102	110
Task 1	126	108	116
Task 2	110	94	102
Task 3	39	33	36
Task 4	32	27	29
Task 5	88	75	81
Task 6	49	42	45
Task 7	110	94	102
Task 8	40	34	37
Task 9	100	86	93
Task 10	105	90	97
Task 11	53	45	49
Task 12	30	26	28
Task 13	28	24	26
Task 14	74	63	68
Task 15	105	90	97
Task 16	70	60	65
Task 17	58	50	54

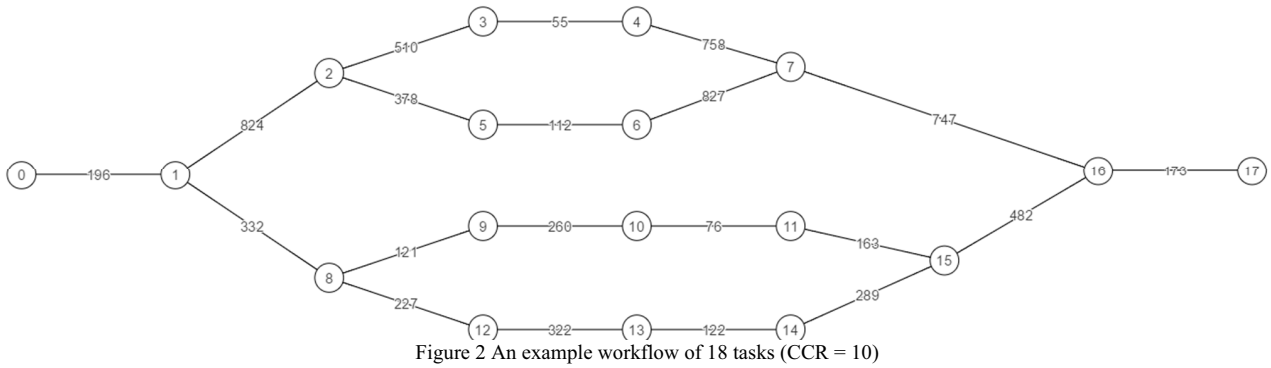


Figure 2 An example workflow of 18 tasks (CCR = 10)

Figure 3 Schedule produced by PEFT

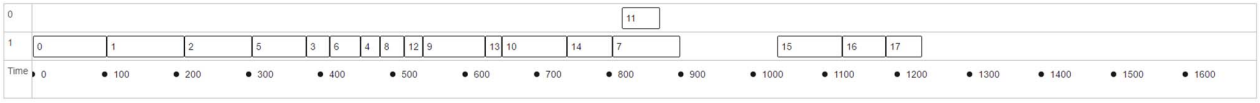


Figure 4 Schedule produced by min-max OCT

Figure 5 Schedule produced by allocated top rank and max-min OCT

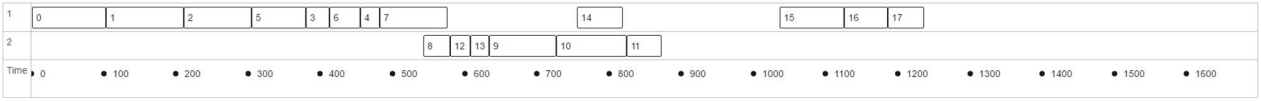


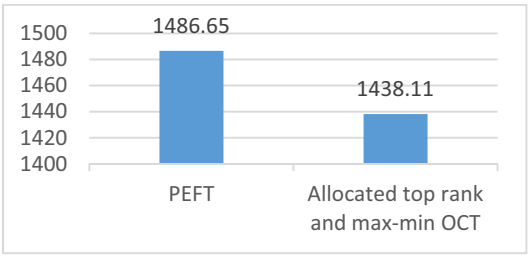
Figure 6 Schedule produced by allocated top rank and min-max OCT

IV. PERFROMANCE EVALUATION

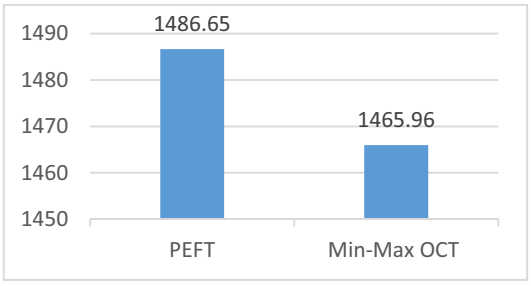
This section presents performance evaluation of the proposed task ranking and allocation heuristics in comparison with PEFT [1] based on a series of simulation experiments. In each experiment, the average makespan of 100 different workflows is used to evaluate different scheduling methods, where makespan measures the total execution time of a workflow. We implemented a DAG generator to randomly generate synthetic workflows for the simulation experiments. The synthetic workflows are to be scheduled on three processors in the experiments.

Figure 7 shows the experimental results for workflows of high CCR values, i.e. 10. Figures 7(a), 7(b), and 7(c) compare the three variants of our heuristics, i.e. allocated top rank + max-min OCT, min-max OCT, allocated top rank + min-max OCT, to PEFT [1], respectively. The experimental results indicate that all the three variants of our heuristics outperform PEFT [1], and the allocated top rank + max-min OCT heuristic achieves the largest performance improvement.

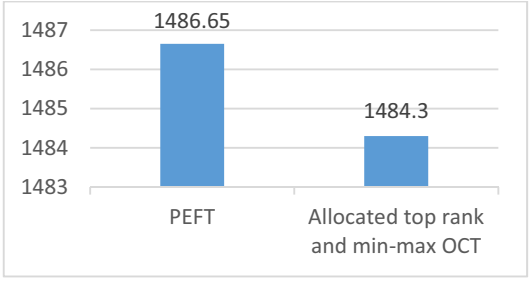
Figure 8 presents the results of another series of simulation experiments, similar to Figure 7 but dealing with workflows of lower CCR values, i.e. 0.1. The experimental results show that the workflow schedules produced by our scheduling heuristics are not as good as what PEFT [1] generates. This is because the advantage of our heuristics, such as allocated top rank, partially lies in more accurate evaluation of inter-task communication costs. Therefore, such advantage is more effective when scheduling workflows of high CCR values.



(a)

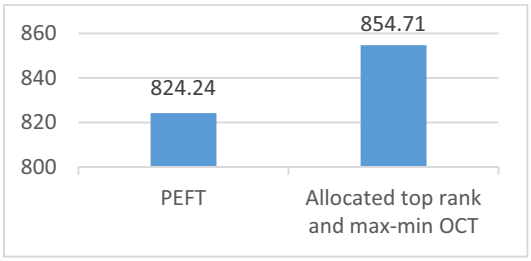


(b)

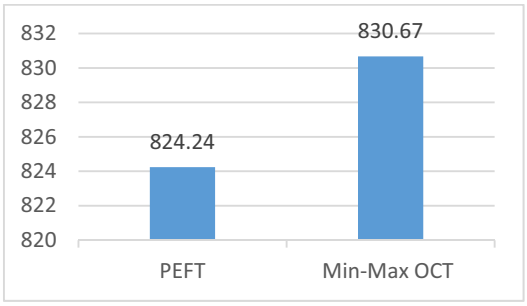


(c)

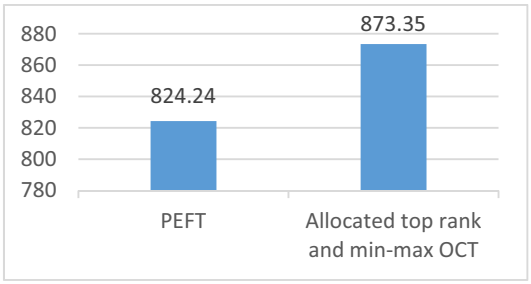
Figure 7 Experiments for workflows with CCR=10



(a)



(b)



(c)

Figure 8 Experiments for workflows with CCR=0.1

V. CONCLUSIONS

Scheduling has long been an important research topic, e.g. [15]. Among various scheduling problems, workflow scheduling [1][2][3][14] is a challenging NP-complete problem and thus many scheduling heuristics have been proposed for tackling it. List-based methods are the most widely used workflow scheduling approaches because of their simplicity and broad applicability. Task ranking and allocation mechanisms are crucial for the effectiveness and performance of list-based workflow scheduling. This paper evaluates various possibilities of task ranking and allocation. We first explore two new task ranking and allocation heuristics, allocated top rank and min-max OCT. Based on these two heuristics, three new workflow scheduling approaches are proposed, including min-max OCT, allocated top rank + max-min OCT, and allocated top rank + min-max OCT. A series of simulation experiments have been conducted to evaluate the proposed approaches and compare them to the state-of-the-art method, i.e. PEFT [2]. Experimental results indicate that effectiveness of the scheduling approaches largely depends on workflows' characteristics. Our approaches are promising in generating better schedules for workflows of higher CCR values, but do not perform as good as PEFT [2] for workflows of lower CCR values. This reflects that further research is needed to develop new task ranking and allocation heuristics for consistently achieving better performance across different workflow characteristics and computing environments.

REFERENCES

[1] H. Arabnejad and J. G. Barbosa, "List Scheduling Algorithm for Heterogeneous Systems by an Optimistic Cost Table", IEEE Transactions on Parallel and Distributed Systems, vol. 25, no. 3, pp. 682-694, March. 2014.

[2] L. F. Bittencourt, R. Sakellariou and E. R. M. Madeira, "DAG scheduling using a lookahead variant of the heterogeneous earliest finish time algorithm", Proc. 18th Euromicro Conference on Parallel, Distributed and Network-based Processing, pp. 27-34, 2010.

[3] K. C. Huang, Y. L. Tsai and H. C. Liu, "Task ranking and allocation in list-based workflow scheduling on parallel computing platform", The Journal of Supercomputing, vol. 71, iss. 1, pp. 217-240, 2015.

[4] Y. L. Tsai, H. C. Liu and K. C. Huang, "Adaptive dual-criteria task group allocation for clustering-based multi-workflow scheduling on parallel computing platform", The Journal of Supercomputing, vol. 71, iss. 10, pp. 3811-3831, 2015.

[5] O. Sinnen, Task Scheduling for Parallel Systems, John Wiley, 2007.

[6] H. Topcuoglu, S. Hariri, M. Y. Wu, "Performance-effective and low-complexity task scheduling for heterogeneous computing", IEEE Transactions on Parallel and Distributed Systems, vol.13 no.3, pp. 260-274, 2002.

[7] L. F. Bittencourt, E. R. M. Madeira, "A performance-oriented adaptive scheduler for dependent tasks on grids", Concurrency and Computation: Practice and Experience, vol. 20, pp. 1029–1049, 2008.

[8] M. Wieczorek, R. Prodan, A. Hoheisel, M. Wieczorek, R. Prodan, A. Hoheisel, "Taxonomies of the multi-criteria grid workflow scheduling problem", Grid Middleware and Services, pp. 237-264, 2008.

[9] Y. L. Tsai, K. C. Huang, H. Y. Chang, J. Ko, E. T. Wang, C. H. Hsu, "Scheduling multiple scientific and engineering workflows through task clustering and best-fit allocation", Proc. IEEE Eighth World Congress on Services, pp. 1-8, June 24- 29, 2012.

[10] T. N'takpe', F. Suter, "A comparison of scheduling approaches for mixed-parallel applications on heterogeneous platforms", Proc. the 6th International Symposium on Parallel and Distributed Computing, 2007.

[11] G. Park, B. Shirazi, and J. Marquis, "DFRN: a new approach for duplication based scheduling for distributed memory multi-processor systems", Proc. International Conference of Parallel Processing, pp. 157-166, 1997.

[12] J. Liou, M. A. Palis, "An efficient clustering heuristic for scheduling DAGs on multiprocessors", Proc. the 8th Symposium on Parallel and Distributed Processing, 1996.

[13] L. F. Bittencourt, E. R. M. Madeira, "Towards the scheduling of multiple workflows on computational grids", J. Grid Computing, vol. 1, no. 8, pp. 419-441, 2009.

[14] J. Thaman and M. Singh, "Extending dynamic scheduling policies in WorkflowSim by using variance based approach", International Journal of Grid and High Performance Computing, vol. 8, no. 2, pp. 76-93, 2016.

[15] C. H. Hsu, T. L. Chen, C. T. Yang and H. C. Chu, "Scheduling contention-free broadcasts in heterogeneous networks", International Journal of Communication Systems, vol. 28, iss. 5, pp. 952-971, 2015.